UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA
COORDINACIÓN DE COMPUTACIÓN

BASES DE DATOS

Tarea 8

Tipos de datos en PostgreSQL

Nombre del alumno: Yukioayax Canek Gabriel Hernández
Profesor: Fernando Arreola Franco
Grupo: 1
Semestre en curso: 2026-2

Instrucciones

Investigar los siguientes tipos de datos en PostgreSQL:

- Numéricos
- Caracter
- Fecha
- Otros tres que llamen la atención

Incluir ejemplos, capacidad y formato de representación.

Introducción

PostgreSQL es un sistema de gestión de bases de datos relacional orientado a objetos que destaca por su robustez, extensibilidad y estricto apego a los estándares SQL, una de sus mayores fortalezas es la amplia variedad de tipos de datos que soporta de manera nativa, que van desde los tipos básicos (numéricos, texto, fechas) hasta tipos más complejos como JSON, arrays y geométricos.

Tipos de datos numéricos

PostgreSQL ofrece varios tipos numéricos que se diferencian por su precisión, rango y uso de almacenamiento.

`INTEGER (INT)`

- **Capacidad:** Números enteros con signo de 4 bytes, rango $-2\,147\,483\,648$ a $2\,147\,483\,647$.
- **Formato de representación:** Secuencia de dígitos opcionalmente precedida por un signo $+$ o $-$.
- **Ejemplo:**

```
CREATE TABLE empleados (
    id INTEGER PRIMARY KEY,
    edad INTEGER
);
INSERT INTO empleados VALUES (1, 28);
```

BIGINT

- **Capacidad:** Entero con signo de 8 bytes, rango: $-9\,223\,372\,036\,854\,775\,808$ a $9\,223\,372\,036\,854\,775\,807$.
- **Formato:** Igual que `INTEGER`.
- **Ejemplo:** Útil para contadores muy grandes o identificadores globales.

SMALLINT

- **Capacidad:** Entero con signo de 2 bytes, rango $-32\,768$ a $32\,767$.
- **Ejemplo:** Edad o cantidad de productos pequeños.

NUMERIC(precision, scale) o DECIMAL

- **Capacidad:** Hasta 131072 dígitos antes del punto decimal y 16383 dígitos después.
- **Formato:** Número con punto decimal, ej. `12345.6789`.
- **Ejemplo:**

```
precio NUMERIC(10,2)  -- máximo 99999999.99
INSERT INTO productos (precio) VALUES (1999.99);
```

REAL y DOUBLE PRECISION

- **Capacidad:** `REAL`: 4 bytes (precisión $\geq$ 6 dígitos decimales). `DOUBLE`: 8 bytes (precisión $\geq$ 15 dígitos).
- **Formato:** Notación decimal o científica, ej. `3.14159` o `6.02E23`.
- **Ejemplo:** Mediciones científicas o financieras que no requieren exactitud decimal exacta.

SERIAL (y BIGSERIAL, SMALLSERIAL)

- **Capacidad:** `SERIAL` es un `INTEGER` con autoincremento; `BIGSERIAL` usa `BIGINT`.
- **Formato:** Se define en la columna y PostgreSQL crea una secuencia implícita.
- **Ejemplo:**

```
CREATE TABLE pedidos (
    id SERIAL PRIMARY KEY,
    descripcion TEXT
);
```

Tipos de datos de caracter

`CHAR(n)` o `CHARACTER(n)`

- **Capacidad:** Cadena de longitud fija de `n` caracteres, si se inserta una cadena más corta, se rellena con espacios.
- **Formato:** Texto entre comillas simples, ej. `'ABC'`.
- **Ejemplo:**

```
codigo_pais CHAR(2) -- 'MX', 'US'
```

`VARCHAR(n)`

- **Capacidad:** Cadena de longitud variable con un máximo de `n` caracteres.
- **Formato:** Texto entre comillas simples, sin relleno de espacios.
- **Ejemplo:**

```
nombre VARCHAR(100)
```

`TEXT`

- **Capacidad:** Cadena de longitud variable sin límite explícito (límite teórico de 1 GB, pero limitado por memoria/disco).
- **Formato:** Texto entre comillas simples.
- **Ejemplo:** Ideal para comentarios, descripciones largas o contenido HTML.

Tipos de datos de fecha y hora

`DATE`

- **Capacidad:** Almacena una fecha (año, mes, día) entre 4713 a.C. y 5874897 d.C.
- **Formato:** PostgreSQL acepta varios formatos, el recomendado ISO es `'YYYY-MM-DD'`.
- **Ejemplo:**

```
fecha_nacimiento DATE DEFAULT '2000-01-01'
```

TIME [(*p*)] [WITH TIME ZONE]

- **Capacidad:** Hora del día con precisión de microsegundos (hasta 6 dígitos fraccionales). Rango: `00:00:00` a `24:00:00-1µs`.
- **Formato:** `'HH:MM:SS[.micro]'` o con zona horaria `'HH:MM:SS+HH:MM'`.
- **Ejemplo:** `hora_entrada TIME DEFAULT '08:00:00'`.

TIMESTAMP [(*p*)] [WITH TIME ZONE]

- **Capacidad:** Combina fecha y hora. `TIMESTAMPTZ` almacena en UTC y convierte según la zona horaria de la sesión.
- **Formato:** `'YYYY-MM-DD HH:MM:SS[.micro]'`.
- **Ejemplo:**

```
fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

INTERVAL

- **Capacidad:** Periodo de tiempo, almacena años, meses, días, horas, minutos, segundos.
- **Formato:** `'1 year 2 months 3 days 04:05:06'`.
- **Ejemplo:** Cálculo de fechas futuras:

```
SELECT CURRENT_DATE + INTERVAL '30 days';
```

Otros tres tipos de datos interesantes

1. JSON / JSONB

- **Descripción:** Almacena datos en formato JSON (JavaScript Object Notation). `JSON` conserva el texto exacto; `JSONB` es una representación binaria optimizada que permite indexación.
- **Capacidad:** Hasta 1 GB (límite práctico). `JSONB` soporta operadores como `@>` (contiene).
- **Formato:** Cadenas JSON válidas, ej. `'"nombre": .ᴬna", .ᵉᵈad": 30'`.
- **Ejemplo:**

```
CREATE TABLE configuracion (
    id SERIAL,
    datos JSONB
);
INSERT INTO configuracion (datos)
VALUES ('{"tema": "oscuro", "idioma": "es"}');
-- Consultar usando el operador de existencia
SELECT * FROM configuracion WHERE datos @> '{"tema": "oscuro"}';
```

2. ARRAY

- **Descripción:** Permite almacenar vectores (arreglos) de cualquier tipo base, incluso multidimensionales.
- **Capacidad:** Limitado por el tamaño máximo de fila (1.6 TB).
- **Formato:** Representación entre llaves, ej. `'1,2,3'` o `'1,2,3,4'` para matrices.
- **Ejemplo:**

```
CREATE TABLE alumno (
    nombre TEXT,
    calificaciones INTEGER[]
);
INSERT INTO alumno VALUES
('Carlos', '{85, 90, 78}');
-- Acceder a un elemento: calificaciones[2]
```

3. BOOLEAN

- **Descripción:** Almacena valores lógicos verdadero/falso/desconocido (`NULL`).
- **Capacidad:** Ocupa 1 byte.
- **Formato:** Se pueden usar múltiples literales: `TRUE`, `'t'`, `'true'`, `'yes'`, `'on'`, `'1'` para verdadero; `FALSE`, `'f'`, `'false'`, `'no'`, `'off'`, `'0'` para falso.
- **Ejemplo:**

```
CREATE TABLE tareas (
    descripcion TEXT,
    completada BOOLEAN DEFAULT FALSE
);
INSERT INTO tareas VALUES ('Estudiar PostgreSQL', 'no');
```

Conclusión

PostgreSQL provee una extensa colección de tipos de datos que se adaptan a prácticamente cualquier dominio de aplicación, los tipos numéricos ofrecen desde enteros pequeños hasta decimales de precisión arbitraria; los tipos de caracter cubren desde códigos fijos hasta documentos completos; y los tipos fecha/hora manejan zonas horarias y operaciones temporales complejas; además, la inclusión de tipos no convencionales como `JSONB`, `ARRAY` y `BOOLEAN` demuestra la flexibilidad del sistema y su capacidad para abordar requerimientos modernos de desarrollo de software sin necesidad de extensiones externas.
Conocer las capacidades y formatos de estos tipos permite diseñar esquemas de base de datos más eficientes y expresivos, aprovechando al máximo las funcionalidades nativas de PostgreSQL.

Referencias

[1] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation: Chapter 8. Data Types*, [En línea]. Disponible en: `https://www.postgresql.org/docs/16/datatype.html`.

[2] B. Momjian, *PostgreSQL: Introduction and Concepts*. Boston, MA, USA: Addison-Wesley, 2001.

[3] R. Obe y L. Hsu, *PostgreSQL: Up and Running*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.